# Object-Oriented Programming

## Collections and Sorting

Aslak Johansen asjo@mmmi.sdu.dk
Amir Ghomashi amgh@mmmi.sdu.dk

November 6, 2026

SDU

# Part 0:
# The `Collections` Namespace

# The `Collections` Namespace ▷ Overview

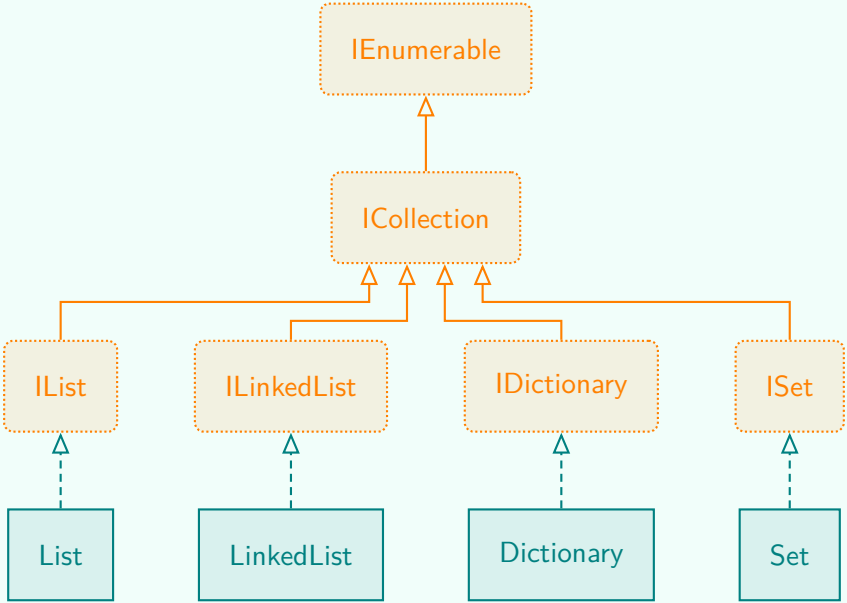
The `Collections` namespace is a part of the .NET Framework.

This namespace describes (through interfaces) a number of types of data structures, including:

1. `System.Collections.Generic.IEnumerable` is used to describe that something can be iterated over.
2. `System.Collections.Generic.ICollection` is used to describe finite collections of elements.
3. `System.Collections.Generic.IList` is used to describe data structures that are indexed. That is, that elements can be accessed through an integer index.
4. `System.Collections.Generic.IDictionary` is used to describe data structures where elements can be accessed through a *"key"*: $key \mapsto value$
5. `System.Collections.Generic.ISet` is used to describe sets. That is, where all elements are unique.

The namespace also contains a number of classes that implement these interfaces (with the described behaviour).

# The `Collections` Namespace ▷ Hierarchy



# Part 1:
# The `ICollection` Interface

# The ICollection Interface

A *"collection"* is a data structure that contains a collection of elements and implements the `ICollection` interface.

The elements are often (but not always) *"ordered"*, i.e. they are arranged in a specific order so that a specific element can be identified by its position in the sequence.

`ICollection` interface contains a number of method prototypes. These include prototypes for:

1. `Add` is used to ensure that the collection contains a specific element.
2. `Remove` is used to remove *a single* reference to a specific object.
3. `Clear` is used to remove all elements from the collection.

# The `ICollection` Interface ▷ Types of Collections

There are a number of different collections in the `Collections` namespace:

- `ISet` Data structures containing a group of unique elements.
- `IList` Data structures containing an ordered group of elements.
- `ILinkedList` Data structures containing a chain of elements.
- `Stack` Data structure that orders elements according to last-in/first-out.
- `Queue` Data structure that orders elements according to first-in/first-out.
- `PriorityQueue` Data structure that orders elements by priority.

We can make more ourselves.

How is that possible?

By implementing the `ICollection` interface!

# Part 2:
# Lists

# Lists ▷ Array Problem

A data structure is an organised collection (not the .NET term) of related data.

The array is the simplest type of data structure in C#.

**Problem:** The size cannot be changed.
**Solution:** Create a new array that is larger than the old one. Copy the contents from the old one into the new one. Update all references.

This has already been implemented through `List` that implements the `IList` interface:

- It is a *dynamic* array. Dynamic because it automatically adjusts its size so that it appears that there is no wasted space.
- Belongs to the .NET Framework (which we will discuss later).
- The user has control over where in the sequence an element is inserted.

# Lists ▷ The IList<T> Interface

The `IList<T>` interface describes a type of collection that is *ordered* and *indexable*.

This means that elements are arranged in a sequence, and that we can determine what should be located where:

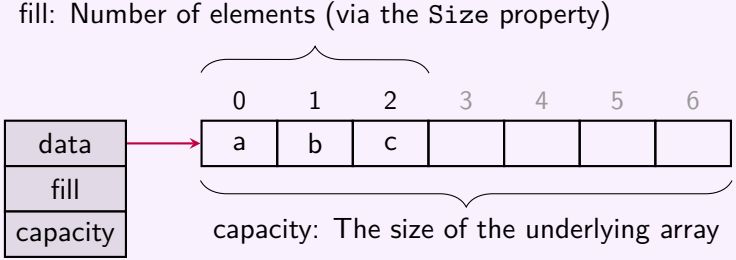
- Properties:
  - `int Count`
  - `T[] Item`
- Methods:
  - `void Insert (int index, T element)`
  - `void RemoveAt (int index)`
  - `int IndexOf (T element)`

There are several classes in the `Collections` namespace that implement this interface, but the most important one is `List<T>`.

# Lists ▷ The `List<T>` Class

Stores elements in an array. Ensures that the array is replaced with a new (larger) array if the capacity of the original is exceeded, or a new (smaller) one if there ends up being too much wasted space.

Used when *random access* is the typical use, meaning when elements that could be located far from both ends of the list are most often accessed ones.



# Lists ▷ Usage ▷ Creation

List can be used for all data types.

```
List<Student> students = new List<Student>();
List<string>  strings  = new List<string>();
```

Only objects of the specified type (and subtypes thereof) can be stored in a `List`.

If you try to add an object of a different type (one that is not more specific), the program cannot be compiled.

# Lists ▷ Usage ▷ `Object`-Parameterization

You could then choose to do the following:

```
List<Object> list = new List<Object>();
```

Since all classes inherit from `Object` all objects can be stored in `list`.

However, this is rarely done. Why is that?

Because then we only have access to the methods defined on `Object` on the elements of `list`.

# Lists ▷ Usage ▷ Generic Types

`List<T>` is a *generic* (or *parameterised*) type.

```
List<T> list = new List<T>();
```

Example:

```
List<string> list = new List<string>();
```

# Lists ▷ Usage ▷ Important Instance Methods

```
void Add (T obj);              //  add item to the end of the list

bool Contains (T obj);         // returns true if obj is in the list

void Insert (int N, T obj); // insert element at N'th position, and shift

void RemoveAt (int N);         // remove element at N'th position
                               // (subsequent elements are moved to
                               // fill the gap)

bool Remove (T obj);           // if a reference to obj is found in the list
                               // remove (first) such reference

int IndexOf (T obj);           // if a reference to obj is found in the list
                               // returns the index of the first such
                               // reference
```

These methods use `Equals`

# Lists ▷ Usage ▷ Traversal

We can traverse a `List<T>` just as we would with an array, but we need to use the `Count` and `Item` attributes.

With a `for` loop (and access through the index):

```
List<int> list = GetRandomList():

for (int i=0 ; i<list.Count ; i++) {
  list.Items[i] += i;
}
```

Or with a foreach loop (without access to the index):

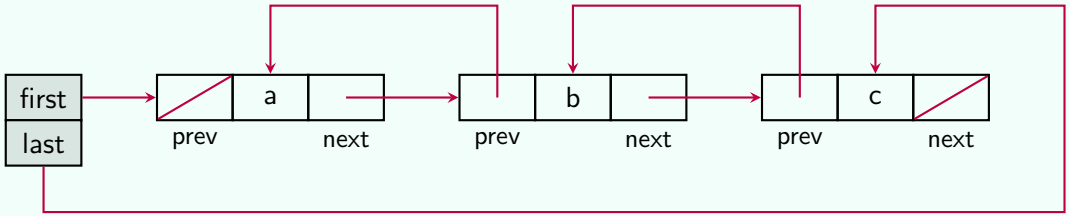
```
foreach (int item in list) {
  Console.WriteLine(item);
}
```

# Part 3:
# Linked Lists

# Linked Lists ▷ Class `LinkedList`

Stores elements in a *chain*, where elements *easily* can be added and removed from the beginning or end.

Used when we need to access elements in sequence from the beginning or end, or add/remove elements from one of the ends.



# Part 4:
# Comparison

# Comparison ▷ Introduction

A number of the things that we might be interested in doing with `Collection` objects require us to have a flexible way of comparing two elements.

**Example:** Sorting a list involves making a lot of comparisons.

# Comparison ▷ The IComparable Interface

Another of the many interfaces in the .NET framework.

Can be used to sort elements in, for example, a `List<T>`.

Comparable defines one method:

```
public int CompareTo (T other);
```

What does it do?

- Return value is negative when the current object is less than `other`.
- Return value is positive when the current object is greater than `other`.
- Return value is 0 when the current object is at level with `other`.

# Comparison ▷ The IComparable Interface ▷ Example (1/2)

```
public class Person : IComparable<Person> {
  private int age;

  public Person (int age) {
    this.age = age;
  }

  public int GetAge () {
    return this.age;
  }

  public int CompareTo (Person? t) {
    return this.age - (t==null ? 0 : t.GetAge());
  }

  public override String ToString () {
    return "" + this.age;
  }
}
```

Alternative way to write CompareTo:

```
public int CompareTo (Person? t) {
  int otherAge = (t==null
                  ? 0
                  : t.GetAge());

  if (this.age > otherAge){
    return 1;
  } else if (this.age < otherAge){
    return -1;
  }
  return 0;
}
```

# Comparison ▷ The IComparable Interface ▷ Example (2/2)

```
public class Program
{
  public static void PrintList (List<Person> persons) {
    Console.Write("List<Person> [");
    for (int i=0 ; i<persons.Count ; i++) {
      Console.Write((i==0 ? "" : ",")+persons[i]);
    }
    Console.WriteLine("]");
  }

  public static void Main ()
  {
    List<Person> persons = new List<Person>();
    persons.Add(new Person(72));
    persons.Add(new Person(4));
    persons.Add(new Person(17));
    PrintList(persons);
    persons.Sort();
    PrintList(persons);
  }
}
```

Output:

```
List<Person> [72,4,17]
List<Person> [4,17,72]
```

If classes had to implement the `IComparable` interface in order to be sorted, this would impose a few limitations:

- Since we have not created all the classes ourselves, it is not always possible for us to ensure that they implement `IComparable`, and thus it would not be possible to sort elements of arbitrary types.
- We would only be able to sort in one way.

# Comparison ▷ The `IComparer` Interface

The `IComparer` interface is implemented by classes whose purpose is to compare objects of a different type.

For instance, using the `IComparer`-interface, we can define a class whose purpose is to compare `string`-objects.

The `IComparer` interface helps us overcome the challenges posed by the `IComparable`:

- We are not dependent on being able to change the definition of the classes whose objects we want to compare.
- We can implement as many `IComparer`-classes as we want for a given type.

# Comparison ▷ IComparer ▷ Example

```
public class DoubleComparer : IComparer<double>
{
  public int Compare (double s1, double s2)
  {
    return (int)(s1 - s2);
  }
}

public class Program
{
  public static void Main ()
  {
    List<double> stringList = new List<double>();
    // fill it up
    stringList.Sort(new DoubleComparer());
  }
}
```

# Part 5:
# Maps

# Maps ▷ Introduction

The `IDictionary` interface defines types of data structures where *values* can be accessed through *keys*.

**Note:** A *"dictionary"* is the same as a *"map"*.

This is somewhat similar to a list where you refer to a value through some object's identity (or rather: hash code) instead of indexing with an integer.

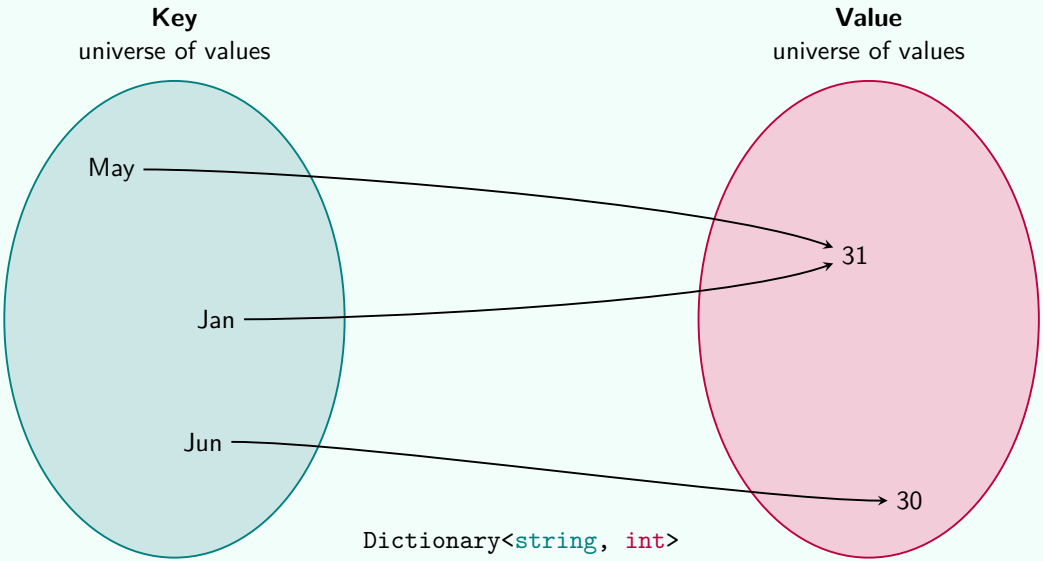
- This code is generated via a call to `GetHashCode` which is defined on `Object`.
- **Note:** If you override `Equals`, then you must also (consider) overriding `GetHashCode`: Two objects that are "equal" must have the same hash code.

There is no check for this. Why?

Rules:

- One key can have one associated value.
- Multiple keys can be associated with the same value.
- You can ask whether a key is associated with a value.

# Maps ▷ Mapping from Keys to Values



# Maps ▷ The `IDictionary` Interface

`IDictionary` defines a number of methods, including:

- `void Add (K key, V value)`
- `void Clear ()`
- `bool ContainsKey (K key)`
- `bool Remove (K key)`

`IDictionary` has a number of properties, including:

- `int Count`
- `V Item[K]`

Since a key can only point to a single value, `Add` will update existing *bindings*.

# Maps ▷ The Dictionary Class

Dictionary is the most commonly used type of map.

With a Dictionary, you can relatively quickly find a reference to a value using a key.

```
Dictionary<string, Person> map = new Dictionary<string, Person>();
Person p = new Person();
map.Add("Aslak", p);
Person fromMap = map["Aslak"];
```

# Part 6:
# Sets

# Sets ▷ Introduction

The `ISet` interface defines types of data structures in which *values* can exist at most once. This is what we call a *"set"* in English.

The `HashSet` class implements this interface and – like `Dictionary` – uses a hash code to distinguish between object identities.

> *Therefore, the previous rule about overriding `GetHashCode` also applies if it should be possible to place an object of the type in question in a `HashMap`. And since it is very easy to overlook, and we do not know how our classes will be used in the future, we should always adjust `GetHashCode` when we tinker with `Equals`.*

# Sets ▷ The ISet Interface

The `ISet` interface describes a type of collection that represents a set of objects.

`ISet` defines, among other things, the following instance methods (which are *destructive*):

- `bool Add (T element)`
- `void Clear ()`
- `bool Contains (T element)`
- `bool IsSubsetOf (IEnumerable other)`
- `bool IsSupersetOf (IEnumerable other)`
- `bool Overlaps (IEnumerable other)`
- `void IntersectWith (IEnumerable other)`
- `void UnionWith (IEnumerable other)`

`ISet` has a number of properties, including:

- `int Count`

# Part 7:
# Sorting

# Sorting

There are several *"recipes"* for sorting the elements in a list. Such recipes are called algorithms.

Which algorithm is best depends on a number of factors. These include:

- Are there many elements of equal size?
- To what extent are the elements already sorted?
- How many elements need to be sorted?

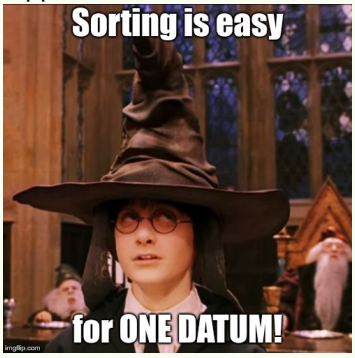
It is **not** part of the curriculum to teach you how to choose the best sorting algorithm for your purpose.

But we need to understand what a sorting algorithm might look like and why it works. We will use "insertion sort" as an example of a sorting algorithm.

*Insertion sort* is a sorting algorithm that is simple to understand and implement.

The main loop of the algorithm iterates through the indices (except for the first one) and ensures that all elements up to this point are sorted.

Q: Why is the first element skipped?




A: Because a sequence

# Sorting ▷ Insertion Sort ▷ Algorithm

Inputs:

- `list`: An object of type `List`
- `n`: Number of elements in `list`

Iterate through all indices `i` from 1 onwards (to `n`-1). For each iteration:

- ***Loop Precondition:*** All elements in `list` up to (but not including) `i` are sorted.
- ***Loop Postcondition:*** All elements in `list` up to and including `i` are sorted.

1. `element` ← `list[i]`
2. Search backwards from `i` for the correct position of `element`. The search is an iteration of `j` from `i`−1 towards 0.
   - ***Loop Postcondition:*** The elements in the subsequence [`j`+1;`i`] are sorted.
   - ***Loop Postcondition:*** Element `j` may be overwritten.

   **If** `list[j]` > `element`:

   `list[j+1]` ← `list[j]`

   **Else**:

   Break out of the loop by jumping to point 3.
3. `list[j+1]` ← `element`

https://www.youtube.com/watch?v=ROalU379l3U

# Sorting ▷ Insertion Sort ▷ Implementation

```
// sort the array arr into increasing order
public int[] InsertionSort (int[] arr) {
  // iterate over all items that may need relocation
  for (int itemsSorted=1 ; itemsSorted<arr.Length ; itemsSorted++) {
    int temp = arr[itemsSorted]; // the item to be inserted
    int loc = itemsSorted - 1;   // start at end of the ordered list

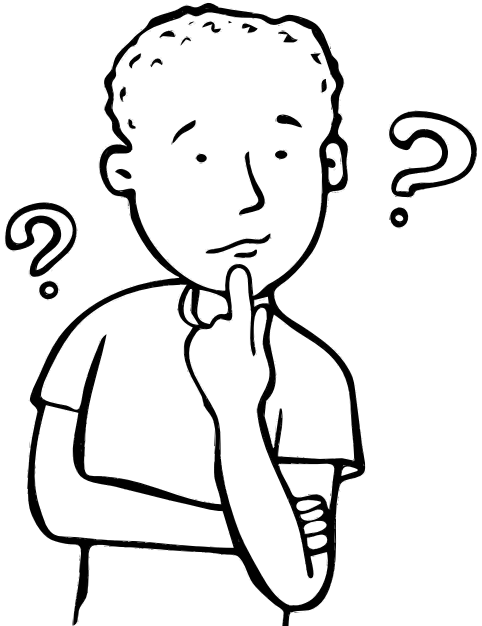
    // move all items larger than temp one spot to the right
    while (loc>=0 && arr[loc]>temp) {
      arr[loc+1] = arr[loc]; // bump item from arr[loc] up to loc+1
      loc = loc - 1; // go on to next location
    }

    // put temp in last vacated space
    arr[loc+1] = temp;
  }

  return arr;
}
```


YOU'RE SLOW
QUICK SORT
MERGE SORT
HEAP SORT
SELECTION SORT
MERGE SORT
QUICK SORT
HEAP SORT
SELECTION SORT
US TOO
SELECTION SORT
BUBBLE SORT
INSERTION SORT

# Questions?



https://openclipart.org/detail/238087/boy-thinking-of-question